# CitizenCare — Test Case Sheet

Generated: 2026-04-26 14:34
Legend: P = Plain text | C = Cipher text | K = Key (not used here) | AI = YOLO detection | RT = Real-time (Socket.io)

| Test Case ID | Test Case Objective | Pre-requisite | Steps | Input Data | Expected Output | Actual Output | Status |
|---|---|---|---|---|---|---|---|
| TC_01 | Backend health check | Server running | 1. Open /api/health<br>2. Observe response | URL: /api/health | { ok: true } | | |
| TC_02 | Python AI health check | ai-model Flask running | 1. Open http://127.0.0.1:5050/health<br>2. Observe response | URL: /health | ok=true, model name, class list | | |
| TC_03 | Citizen registration | MongoDB connected | 1. Open Register<br>2. Enter name/email/password<br>3. Submit | name/email/password | Account created; login possible; role=citizen | | |
| TC_04 | Police login (seed user) | Run server seed (npm run seed) | 1. Open Login<br>2. Enter police creds<br>3. Submit | police@citizencare.local / Police12345! | Login success; redirect to police dashboard | | |
| TC_05 | Citizen submits a crime report | Citizen logged in | 1. Open Report<br>2. Fill title/desc/type<br>3. Set lat/lng<br>4. Submit | crimeType, lat/lng, title | Report created with status=pending | | |
| TC_06 | Police verifies report (RT notify) | Police logged in; pending report exists | 1. Open Incidents<br>2. Open a pending report<br>3. Set status=verified | reportId, status=verified | Status updated; map shows marker; citizen gets notification; RT public-safety emitted | | |
| TC_07 | CCTV device selection shows Camo camera | Camo Studio running; browser permission allowed | 1. Open CCTV AI page<br>2. Refresh devices<br>3. Select Camo camera | Video device list | Camo camera appears in dropdown and is selectable | | |
| TC_08 | CCTV stream starts and uploads 1 FPS | Backend + AI running; user logged in | 1. Start camera<br>2. Wait 10 seconds<br>3. Observe status & backend logs | Live video frames | Camera Connected □; AI Running...; backend logs detections or No objects detected | | |
| TC_09 | AI detects knife on static image test API | Python AI running; user logged in | 1. Call /api/cctv/detect-test<br>2. Upload Knife.jpg<br>3. Check response | image=Knife.jpg, conf=0.15 | detectedClasses contains knife; weaponDetected=true | | |
| TC_10 | Safety suggestion API returns status+score | Backend running; some data optional | 1. Call /api/safety-status?lat&lng<br>2. Observe JSON | lat,lng,radiusM=1000,days=14 | JSON: status SAFE/LOW/HIGH + numeric score + breakdown | | |
| TC_11 | Citizen map click updates safety card | Citizen logged in; map key set | 1. Open citizen Safety tab<br>2. Click on map point<br>3. Observe safety card | Map click lat/lng | Safety card updates (color + score); blue marker shows selected point | | |
| TC_12 | Real-time safety refresh on public-safety event | Citizen logged in; socket connected; police verifies report OR CCTV alert occurs | 1. Keep citizen map open<br>2. Trigger verified report or cctv-alert<br>3. Observe refresh | Socket event: public-safety | Citizen map reloads overlays and recomputes safety status | | |